

REPORT_VITIKA

Autonomous Auction Intelligence Ecosystem & Adaptive Fitness MAS

This report combines the complete project journey. The narrative begins with vehicle valuation intelligence, progresses into autonomous auction decision making and agentic reasoning, and concludes with the Adaptive Fitness Multi-Agent System. The objective is to explain not only what was built, but why it was built, how each component works, and how every phase contributes to the final system.

Section 1

Story-Driven Report Enhancement

My Thought Process

I did not start by building an auction agent.

I first asked a fundamental question:

"How can an autonomous system make intelligent decisions if it does not know the true value of a vehicle?"

Because of that, Phase 1 became the valuation intelligence foundation of the entire ecosystem.

Phase Structure

Phase 1 — Vehicle Price Prediction & Data Intelligence

1.1 Understanding the Dataset

1.2 Cleaning Real Auction Data

1.3 Exploring Market Behaviour

1.4 Engineering Strategic Features

1.5 Training the Valuation Engine

1.6 Evaluating Performance

1.7 Exporting Deployment Artifacts

Autonomous Auction Intelligence Ecosystem

Phase 1 — Vehicle Price Prediction & Data Intelligence

> **"The job isn't just to predict a price. It's to build the brain that everything else will depend on."**

What Is This?

This notebook is **Phase 1** of a multi-phase autonomous auction bidding ecosystem. Before any bidding logic can exist, before any agent can decide whether to compete for a vehicle — there needs to be a reliable answer to one fundamental question: *what is this car actually worth?*

That's what this phase builds. A clean, well-engineered, auction-aware price prediction engine using CatBoost, trained on real used-car auction data from across the United States.

But it's not just a regression model. Every design decision here — from which features to engineer, to how outliers are handled, to how the model is saved — was made with the downstream ecosystem in mind. This is the **valuation intelligence core** that all future strategic decisions will rely on.

The Problem We're Solving

Used-car auction prices are messy. They're affected by:

- How old the car is
- How many miles it's been driven (and how hard)
- What brand it is — and whether that brand carries luxury prestige
- What condition it's in
- Where the auction is happening
- What color it is (yes, that actually matters)

A naive model that ignores all these interactions will produce unreliable estimates. And an unreliable estimate means an agent that either overbids (losing money) or underbids (losing opportunities). Neither is acceptable.

The goal here is a model accurate enough that future phases can *trust* it as a ground truth signal.

Dataset

The training data is ``car_auction_train.csv``, containing historical used-car auction records from across the US.

| Column | Description |

|---|---|

| ``year`` | Manufacture year |

| ``make`` | Brand (Ford, BMW, Toyota...) |

| ``model`` | Specific model name |

| ``trim`` | Variant/trim level (LX, Sport, EX...) |

| ``body`` | Body type (Sedan, SUV, Pickup...) |

| ``transmission`` | Automatic or manual |

| ``state`` | US state where the auction happened |

| ``condition`` | Numeric condition rating |

| ``odometer`` | Mileage at time of sale |

| ``color`` | Exterior color |

| ``interior`` | Interior color |

| ``sellingprice`` | ****Target**** — the final hammer price |

Pipeline Walkthrough

Phase 01 — Library Imports

We use ****CatBoost**** as the sole model. The reason is deliberate: CatBoost handles categorical variables natively, meaning we don't need ``get_dummies()`` or label encoding for columns like ``make``, ``model``, ``body``, etc. This matters because those strings carry real meaning — "bmw" and

"ford" are not just different numbers, they have different market behaviors — and CatBoost can learn that directly.

Other libraries: ``pandas``, ``numpy``, ``sklearn`` (for metrics), ``matplotlib``, ``seaborn``, and ``shap`` for interpretability.

Phase 02 — Data Loading

Data is read from CSV. Simple, but the important part is what comes next: understanding it before touching it.

Phase 03 — Initial Data Analyzation

Before any cleaning, we look at:

- Shape and column names
- Missing value counts across every column

This step exists because you can't make good cleaning decisions without knowing what's actually broken. Jumping straight to ``fillna()`` without inspecting first is how you introduce silent bugs.

Phase 04 — Data Cleaning & Stripping

****Column normalization:**** All column names are lowercased. Sounds trivial, but when preprocessing runs on a single live auction dictionary (which it must, per the task rules), case inconsistencies will break things silently.

****Target cleaning:**** Rows where ``sellingprice`` is missing are dropped entirely. You cannot impute a training target — that would just teach the model to predict your imputation strategy.

****Price outlier removal:**** Prices below the 1st percentile and above the 99th percentile are removed. The justification: prices below ~\$100 or above ~\$150,000 are almost certainly data entry errors or edge cases the model doesn't need to generalize to. Keeping them would distort the loss function without adding meaningful signal.

Phase 05 — Feature Engineering

This is the most important phase. Raw columns tell you facts; engineered features tell you **meaning**. Here's every feature created and why:

``car_age``

`car_age = 2025 - year`

Age matters more than year in most models. A 2018 car bought in 2020 is very different from one bought today. Age captures depreciation more directly.

``age_bucket``

Cars are bucketed into ``new`` (≤ 3 years), ``mid`` (≤ 8 years), and ``old`` (> 8 years). This gives the model a soft categorical signal — it can learn that "old" cars behave differently as a class, not just that higher `car_age` correlates with lower price.

``is_luxury``

Binary flag: 1 if the make is in `{bmw, audi, mercedes-benz, lexus, porsche, jaguar, land rover, cadillac, lincoln}`, else 0.

Why does this matter? Because luxury cars depreciate differently. Their floor prices are higher, their resale confidence is stronger, and the model needs to know this explicitly rather than inferring it purely from the ``make`` column. This feature becomes especially important for the bidding agent in Phase 2 — luxury vehicles get lower required margins precisely because resale risk is lower.

``mileage_per_year``

```
mileage_per_year = odometer / (car_age + 1)
```

A car with 80,000 miles that's 10 years old is in far better shape than one with the same mileage that's 4 years old. Raw odometer doesn't capture this — mileage intensity does.

``condition_bucket``

Condition scores ≥ 4 = ``excellent``, ≥ 2.5 = ``average``, below = ``poor``. Same idea as age bucket — it helps the model reason categorically about quality tiers, not just linearly across a numeric scale.

``is_preferred_color` / `is_preferred_interior``

Neutral colors (black, white, silver, gray) and clean interiors (black, gray, beige, tan) hold resale value better. This is an auction-specific insight: buyers at wholesale auctions know certain colors sell faster on the lot.

``make_score`, `model_score`, `trim_score`, `body_score``

These are **target-encoded** market desirability scores — the median selling price for each category. They compress high-cardinality categorical columns into a single numeric signal that reflects market demand.

For example, ``make_score`` for BMW might be ~\$28,000 while for a budget brand it might be ~\$8,000. This directly encodes brand value into the model in a way CatBoost can use for numeric reasoning.

``market_desirability_score``

A composite blend of ``make_score``, ``model_score``, and ``trim_score``, normalized. This single feature summarizes "how desirable is this exact vehicle in the current market."

``usage_quality_score``

Combines ``mileage_per_year`` and ``condition`` into a single quality signal. Low miles + excellent condition = high score. This feature later feeds into conviction scoring in the bidding engine.

``odometer_clipping``

Extreme odometer values (below 1st percentile, above 99th percentile) are clipped rather than dropped. The data is kept, but extreme values are winsorized to prevent the model from overfitting to edge cases.

Phase 06 — Exploratory Data Analysis

EDA here goes beyond bar charts. The goal is to find *non-obvious* pricing patterns:

- **Depreciation by body type over time:** SUVs depreciate differently than Sedans over a 10-year window
- **Condition threshold effects:** Prices don't fall linearly with condition — there are cliff edges where ratings below a threshold cause sharp drops
- **Odometer-to-year ratio vs. single features:** Shows that usage intensity predicts price better than raw mileage alone
- **Transmission × body type combinations:** Some pairings (e.g., manual Pickup trucks) consistently outperform segment averages

These insights shaped which features were engineered in Phase 05, and they'll inform future auction strategy in Phase 2.

Phase 07 — Feature Selection

After engineering, we select the final feature set for the model. Columns that are redundant, too high-cardinality without encoding, or that would cause data leakage are excluded from `X`.

The target `y` is `log1p(sellingprice)`. Why log-transform?

Because car prices are right-skewed — there are a few very expensive cars and many cheap ones. Training on raw prices makes the model overweight expensive vehicles in its loss function. Log-transforming brings the distribution closer to normal, which CatBoost (and most gradient boosters) prefer. At prediction time, we reverse this with `expm1()`.

Phase 08 — Train/Test Split

80/20 split with `random_state=42` for reproducibility. The 20% test set is held out completely from all training and tuning steps.

Phase 09 — CatBoost Baseline + Cross-Validation

A baseline `CatBoostRegressor` is trained first — no tuning, just default-ish params — to establish a performance floor. This gives us something to compare hyperparameter tuning against.

5-fold cross-validation is then run using CatBoost's built-in `cv()` function (rather than sklearn's wrapper). The reason: CatBoost's CV evaluates using its own ordered boosting internally, which is more accurate than wrapping it in sklearn's generic interface.

The CV plot shows train vs. test RMSE across iterations with ± 1 std shading, plus a vertical line at the best iteration. This visual is important — it confirms the model isn't overfitting.

Phase 10 — Hyperparameter Tuning

CatBoost's built-in `grid_search()` is used to tune:

- `learning_rate`: [0.03, 0.05]
- `depth`: [6, 8]

Why these parameters specifically?

- **Learning rate** controls how aggressively the model corrects errors on each iteration. Too high → overshoots and overfits. Too low → too slow to converge.

- **Depth** controls tree complexity. Deeper trees capture more interactions but risk overfitting on smaller datasets.

The tuned model is retrained from scratch using the best found parameters + early stopping (50 rounds), which prevents running unnecessary iterations after validation RMSE stops improving.

A side-by-side comparison of default vs. tuned model performance is plotted across RMSE, MAPE, and R².

Phase 11 — Model Evaluation

The final model is evaluated on the held-out test set using:

| Metric | What It Tells Us |

|---|---|

| **RMSE** | Average dollar error magnitude (penalizes large errors heavily) |

| **MAE** | Average absolute dollar error (more interpretable) |

| **MAPE** | Average percentage error (scale-independent) |

| **R²** | How much variance the model explains |

Two diagnostic plots are generated:

1. **Actual vs. Predicted scatter** — points should cluster tightly around the diagonal
2. **Residuals plot** — checks for heteroscedasticity (if errors are larger for expensive cars, we have a problem)

Phase 12 — Feature Importance + SHAP

CatBoost feature importance shows which features contributed most to splitting decisions across all trees.

SHAP values go deeper — they show the *direction* of each feature's impact on individual predictions. For example:

- High `is_luxury` → pushes predicted price up

- High `mileage_per_year` → pushes predicted price down

- High `market_desirability_score` → pushes price up

This interpretability matters for the ecosystem because Phase 4's reasoning pipeline needs to understand *why* a car is valued the way it is, not just what the number is.

Phase 13 — Error Analysis

The 20 worst predictions are examined manually. We look at which makes, models, and conditions caused the largest errors. This is where we find out if there are systematic blind spots — maybe the model struggles with rare luxury trims, or with heavily modified vehicles.

Knowing where predictions fail is as important as knowing the average accuracy.

Phase 14 — Model Export

Two files are saved:

`model_VITIKA.cbm` — The CatBoost model in its native binary format. This gets loaded directly into the agent's `__init__`.

`encoders_VITIKA.pkl` — A dictionary containing everything needed to preprocess a single new vehicle record at inference time:

- `cat_features`: list of categorical column names
- `preferred_colors` and `preferred_interiors`: for feature flags
- `luxury_brands`: for `is_luxury`
- `make_score_map`, `model_score_map`, `trim_score_map`, `body_score_map`: target encoding maps
- `feature_columns`: the exact ordered list of columns the model expects
- `odometer_clip`: min/max clip bounds from training
- `global_mean_price`: fallback for missing target-encoding keys

The encoder dict is the critical piece. Without it, preprocessing would have to be re-derived from scratch at inference time, which (a) is impossible for single-row predictions and (b) would produce different results than training.

Why CatBoost?

This wasn't an arbitrary choice. The dataset has many categorical columns (`make`, `model`, `trim`, `body`, `state`, `color`, `interior`) with high cardinality. CatBoost:

1. Handles categorical columns **natively** — no encoding needed during preprocessing
2. Uses **ordered boosting** — reduces overfitting on small samples
3. Has a **built-in grid search** that's more efficient than sklearn wrappers
4. **Symmetric trees** — faster prediction at inference time (important for a live auction agent)

Compared against Random Forest and LightGBM (both were evaluated), CatBoost consistently produced better RMSE and R^2 on this dataset.

Key Strategic Insights from EDA

These findings directly influence future phases:

| Observation | Strategic Impact |

|---|---|

| Luxury brands show lower price volatility | Lower margin requirements in Phase 2 |

| Mileage intensity > raw odometer for pricing | Better conviction scoring in Phase 4 |

| Condition cliff effects at ~2.5 and ~4.0 | Risk flags in Phase 4 risk assessment |

| Preferred colors/interiors add ~3-8% to price | Market desirability scoring in Phase 5 |

| Market desirability score is top-5 feature | Direct input to opportunity scoring |

Files Generated

| File | Purpose |


```
|---|---|
| `model_VITIKA.cbm` | Trained CatBoost model |
| `encoders_VITIKA.pkl` | Preprocessing artifacts for inference |
| `cv_comparison.png` | Cross-validation learning curve |
| `model_comparison.png` | Default vs. tuned comparison |
| `final_model_diagnostics.png` | Actual vs. predicted + residuals |
| `feature_importance.png` | Top-20 feature importances |
| `shap_summary.png` | SHAP value distribution |
---
```

What Comes Next

Phase 1 ends with a trained, validated, export-ready prediction engine. Phase 2 takes that engine and wraps it inside a **strategic bidding agent** — one that doesn't just predict prices, but decides *when to bid, how much, and when to walk away*.

Everything after this depends on getting Phase 1 right.

Quick Start

```
```python
import pickle
from catboost import CatBoostRegressor
```

### Load model

```
model = CatBoostRegressor()
model.load_model("model_VITIKA.cbm")
```

### Load encoders

```
with open("encoders_VITIKA.pkl", "rb") as f:
 encoders = pickle.load(f)
```

## Single-row inference example

**(preprocessing logic lives in the agent's  
analyze\_item method)**

```
```
```

Part of the Autonomous Auction Intelligence Ecosystem | Task 1 — Pawn Stars Bidding Challenge

Section 2

Story-Driven Report Enhancement

Why I Built This

After completing valuation intelligence, I realized that prediction alone does not create profit.

A profitable auction participant must:

- reason,
- remember,
- forecast,
- preserve capital,
- and adapt continuously.

This motivated the development of Phases 2–7.

Complete Architecture

Phase 2 — Elite Strategic Bidding Engine

2.1 Margin Intelligence

2.2 Sigmoid Bid Progression

2.3 Conviction-Based Aggression

2.4 Opportunity Preservation

Phase 3 — Memory-Driven Adaptive Learning

3.1 Episodic Memory

3.2 Similarity Retrieval

3.3 Failure Analysis

3.4 Reward-Based Learning

Phase 4 — Strategic Reasoning Pipeline

4.1 Market Analysis

4.2 Risk Assessment

4.3 Opportunity Cost

4.4 Conviction Scoring

4.5 Dynamic Bid Sizing

4.6 Final Decision

Phase 5 — Multi-Agent Ecosystem

5.1 Luxury Agent

5.2 Margin Hunter Agent

5.3 Risk Agent

5.4 Market Observer Agent

5.5 Opportunity Preservation Agent

5.6 Deliberation Engine

5.7 Communication Hub

Phase 6 — Future Market Intelligence

6.1 Forecasting

6.2 Regime Prediction

6.3 Opportunity Density

6.4 Volatility Forecasting

6.5 Profit Forecasting

6.6 Forecast Driven Adaptation

Phase 7 — Governance Layer

7.1 Health Monitoring

7.2 Strategy Rotation

7.3 Capital Allocation

7.4 Ecosystem Coordination

7.5 Emergency Overrides

7.6 Long-Term Optimization

Autonomous Auction Bidding Ecosystem

Phase 2 → Phase 7 — Strategic Bidding Intelligence

> **"A system that wins every auction is not a good system. A system that wins the right auctions, skips the wrong ones, and survives long enough to compound profits — that is a good system."**

What Is This?

This notebook is everything that happens *after* the price prediction model (Phase 1) is built. Phase 1 answered: *what is this car worth?* This notebook answers the harder question: *what should we actually do about it?*

The result is a fully autonomous bidding agent — `BiddingAgent` — that participates in a live auction simulation managing a \$500,000 budget across thousands of vehicles. It doesn't just predict and bid. It reasons, remembers, adapts, forecasts, collaborates internally, and governs itself.

The architecture spans six phases (Phase 2 through 7), each adding a new layer of intelligence. By the end, the agent behaves less like an algorithm and more like a professional auction trader with memory and strategic discipline.

The Core Philosophy

Most bidding systems optimize for: *win this auction*.

This system optimizes for: *maximize long-term profitability across many auctions*.

That one difference changes everything. It means:

- Skipping bad opportunities, even when winning is possible
- Preserving capital for better future vehicles
- Reducing aggression in overheated markets
- Learning from failures instead of repeating them

The mantra embedded throughout the code: **few elite trades are better than many thin ones.**

System Architecture at a Glance

...

Vehicle Arrives

↓

Phase 1: Price Prediction (CatBoost)

↓

Phase 4: Strategic Reasoning Pipeline

↓

Phase 3: Memory Retrieval (what happened in similar past auctions?)

↓

Phase 6: Future Market Intelligence (is something better coming?)

↓

Phase 5: Multi-Agent Deliberation (5 agents vote)

↓

Phase 7: Governance Validation (is this safe for the ecosystem?)

↓

Phase 2: Bid Computation (sigmoid-decay curve)

↓

Final Bid or Pass

↓

Memory Update + Policy Evolution

...

Everything flows through this pipeline for every single car. Each layer can veto, modify, or approve the bid before it goes out.

Phase 2 — Elite Strategic Bidding Engine

The Problem with Linear Bidding

Most naive bidding agents increment bids by a fixed amount each round. The problem: by the final rounds of an auction, they've exhausted headroom and either overpay (emotional chasing) or drop out too early (missed profits).

Sigmoid-Decay Bid Curves

The bidding engine uses a **sigmoid-decay function** to control bid aggression across rounds:

```
```python
decay = 1.0 / (1.0 + exp(ALPHA * (round_number - MIDPOINT)))
```
```

What this means in practice:

| Auction Stage | Behavior |

|---|---|

| Early rounds (1–3) | Aggressive — large increments to test competition |

| Mid rounds (4–7) | Controlled — increments shrink as price approaches fair value |

| Late rounds (8+) | Conservative — tiny steps, maximum discipline |

This curve mathematically *prevents* irrational final-round overbidding. The later in the auction, the less the system is willing to pay above fair value.

Conviction Scaling

The bid increment isn't just a function of round number — it's also scaled by **conviction**. High-conviction opportunities (where the model is confident the vehicle is undervalued) get larger increments. Low-conviction opportunities get smaller ones.

```
```python
increment = max(headroom * decay * conviction_scale * aggression_scale * mode_scale,
min_increment)
```
```

- `conviction\_scale`: how confident we are this car is a good deal (0–1)

- ``aggression_scale``: globally tuned by Phase 3's policy engine (starts at 1.0, shifts based on recent performance)
- ``mode_scale``: 0.7 (conservative), 1.0 (balanced), 1.3 (aggressive) — set by Phase 7's governor

Luxury Minimum Increments

Luxury vehicles get a higher minimum bid step (\$200 vs. \$50). The reasoning: placing a \$50 bid on a \$40,000 car looks weak, signals low conviction, and invites competitors to escalate cheaply. A larger minimum step projects confidence and forces real commitment from competitors.

Maximum Bid Computation

Before any bidding starts, the system computes a hard ceiling:

```
```python
required_margin = base_margin - luxury_tolerance (if luxury) + regime_adjustment +
memory_adjustment
max_bid = predicted_value * (1 - required_margin)
```
```

If the current highest bid already exceeds ``max_bid``, the system passes immediately. No emotion, no chasing.

Phase 3 — Memory-Driven Adaptive Learning

Why Static Systems Fail

A system with no memory repeats the same mistakes indefinitely. It doesn't know that last Tuesday, in a hot market, luxury sedans consistently sold \$3,000 over prediction. It doesn't know that mid-range SUVs in excellent condition win at lower margins than the model expects.

Phase 3 gives the system a brain that remembers.

The `AuctionRecord` — What Gets Stored

Every auction outcome is stored as a structured record:

```
```python
@dataclass
class AuctionRecord:
 car_id: str
 predicted_value: float
 actual_price: float
 our_final_bid: float
 won: bool
 profit_if_won: float
 margin_pct: float
 market_regime: str # cold/normal/warm/hot/overheated
 auction_stage: str # opening/mid/late/closing
 vehicle_segment: str # economy/mid_range/luxury/exotic
```
```

```

rounds_taken: int
competition_proxy: float # how fast did the price escalate?
regret: float # opportunity cost if we lost a good one
...

```

500 records are kept in a rolling window. Old records expire as new ones come in.

Similarity-Based Retrieval

When a new vehicle arrives, the system doesn't just check raw averages. It searches memory for ****similar past situations****:

```

```python
def retrieve_similar(query_value, query_segment, query_regime, top_k=5):

```

## **Score each memory by price proximity + regime match**

```

price_sim = 1.0 / (1.0 + abs(ep.predicted_value - query_value) / query_value)
regime_bonus = 0.3 if ep.market_regime == query_regime else 0.0
...

```

The top-5 most similar past records are retrieved and used by Phase 4's reasoning nodes to contextualize the current decision.

### ***Failure Analysis***

Every 30 losses are analyzed as a group:

```

```python
failure_analysis = {
    "avg_loss_margin": how far off was our max bid vs. actual price?
    "hot_market_loss_rate": fraction of losses that happened in hot markets?
    "luxury_loss_rate": are we systematically losing luxury auctions?
    "late_stage_loss_rate": are we losing because we run out of headroom late?
}
...

```

This feeds directly into the ``AdaptivePolicyEngine``.

The `AdaptivePolicyEngine` — Policy That Evolves

The policy engine holds the current behavioral state:

Parameter	What It Controls
--- ---	
<code>`base_margin`</code>	Required profit margin before bidding (default 10%)
<code>`aggression_scale`</code>	Global multiplier on bid increments (default 1.0)
<code>`skip_threshold`</code>	How good does conviction need to be before we bid?
<code>`strategy_mode`</code>	<code>`conservative`</code> / <code>`balanced`</code> / <code>`aggressive`</code>

After every auction, the policy updates:

- **Lost a good one?** → Slightly lower the required margin (be more willing to bid next time)
- **Won but with thin margin?** → Raise the required margin (we're cutting it too close)
- **Hot markets keep causing losses?** → Reduce aggression scale (stop fighting overheated auctions)
- **Win rate >60% and bankroll healthy?** → Raise aggression slightly (we can push harder)

This is continuous learning — not from a training set, but from live auction experience.

Phase 4 — Advanced Strategic Reasoning Pipeline

Why One Decision Function Isn't Enough

A single `should_bid()` function can't simultaneously handle market analysis, risk assessment, opportunity cost, and bid sizing. Each concern requires different logic, different inputs, and different outputs.

Phase 4 structures the decision as a **sequential reasoning pipeline** — inspired by LangGraph-style agentic architectures. Each node is a specialist. The context object passes through all of them.

The `AuctionContext` — Shared Decision State

Everything relevant to the current decision is packed into an `AuctionContext``:

```
```python
@dataclass
class AuctionContext:
 car_features: Dict # raw vehicle attributes
 predicted_value: float # from Phase 1 model
 vehicle_segment: VehicleSegment
 bankroll: float
 current_highest_bid: float
 round_number: int
 market_regime: MarketRegime
 auction_stage: AuctionStage
 session_position: float # 0.0–1.0
 memory_records: List[AuctionRecord] # from Phase 3
 forecast: Dict # from Phase 6
```

### **Populated by pipeline nodes:**

```
conviction_score: float # 0–1
max_bid: float # hard ceiling
skip_decision: bool # True = pass on this auction
skip_reason: str
reasoning_trace: List[str] # full audit trail
```
```


Node 1: Market Analysis

Examines `market\_regime` and `auction\_stage`. Hot or overheated markets trigger margin increases. Late auction stages trigger aggression reductions. The output is a market-aware max bid adjustment.

Node 2: Risk Assessment

Checks for red flags:

- Condition below 2.5 (cliff effect identified in EDA)
- Very high mileage-per-year ratio
- Bankroll below 40% of starting value
- Exotic segment with insufficient bankroll buffer

If risk score exceeds threshold, the car is skipped.

Node 3: Opportunity Cost

This is the node that asks: **should we spend money now, or save it for something better?**

It consults Phase 6's `FutureMarketIntelligence` to get `expected\_future\_profit\_score` and `opportunity\_density`. If the forecast says high-quality opportunities are likely coming and the current car's margin is thin, the system skips.

```
```python
opportunity_cost = forecast["expected_future_profit_score"]
if opportunity_cost > current_conviction * 1.2:
 ctx.skip_decision = True
 ctx.skip_reason = "Preserving capital for better future opportunity"
```
```

This is **strategic patience** — one of the most important properties of a professional auction participant.

Node 4: Conviction Scoring

Combines:

- Predicted profit margin vs. required margin (how much buffer do we have?)
- Historical win rate for this segment + regime combination
- Market desirability features from Phase 1
- Phase 6 opportunity density

Output is a single `conviction\_score` between 0 and 1. This directly scales bid aggressiveness in Phase 2.

Node 5: Dynamic Bid Sizing

Converts conviction + max_bid + current_highest_bid into concrete bid constraints that the Phase 2 engine uses.

Node 6: Final Decision

Reviews the full reasoning trace, applies any governance overrides from Phase 7, and produces the final `skip\_decision` and `max\_bid`.

Phase 5 — True Multi-Agent Ecosystem

One Agent Is Never Enough

Any single agent has blind spots. A purely margin-focused agent will miss high-quality luxury opportunities because the margins look thin. A purely aggressive agent will blow its bankroll in an overheated market.

Phase 5 runs **five specialized agents simultaneously**, each evaluating the same vehicle from a different perspective. They vote. The result is a weighted consensus.

The Five Agents

`LuxuryAgent`

Activates only for luxury and exotic vehicles. Applies lower margin tolerances (8% vs. 10%) because luxury vehicles have more stable resale floors. Uses segment-specific win rate from memory to calibrate conviction.

`MarginHunterAgent`

Obsessed with profit margins. Requires a healthy spread between predicted value and current bid. If the margin is thin, it votes no regardless of other factors. This agent is the financial discipline enforcer.

`RiskAverseAgent` ***(has VETO power)***

The risk officer. Checks bankroll ratios, competition intensity, and market regime. If bankroll is below 30% of starting or the market is overheated, it can issue a **hard veto** that overrides all other votes.

```
```python
```

```
VETO_AGENTS = {"RiskAverseAgent"} # these can kill a bid entirely
```

```
```
```

This is intentional — capital preservation is non-negotiable.

`MarketObserverAgent`

Focuses on external signals: Phase 6 market forecasts, competition proxy from recent memory, current regime. Determines whether the *market context* supports bidding right now or suggests waiting.

`OpportunityPreserverAgent`

Advocates for skipping current auctions when the forecast suggests better opportunities are coming. This agent is the one that says "this car is fine, but something better is coming in 10 auctions — save the capital."

Deliberation Process

```
```python
```

```
def deliberate(ctx):
```

## **Step 1: All agents evaluate independently**

```
votes = [agent.evaluate(ctx) for agent in agents]
```

## Step 2: Hard veto check

if RiskAverseAgent voted NO: return (skip, veto\_reason)

## Step 3: Filter abstentions (conviction=0)

active\_votes = [v for v in votes if v.conviction > 0]

## Step 4: Weighted consensus

consensus\_ratio = sum(conviction for bids) / total\_conviction

## Step 5: If consensus\_ratio > 0.5 → bid, else skip

final\_max\_bid = conviction\_weighted\_average(suggested\_max\_bids)

---

The system doesn't just count votes — it weights them by conviction. An agent that's 90% confident has more influence than one that's 40% confident.

### ***`CommunicationHub`***

All agent reasoning is broadcast to a shared hub, creating an audit log of every thought in the deliberation process. This is critical for debugging and also mirrors how real collaborative decision teams work.

---

## Phase 6 — Future Market Intelligence Engine

### ***Why Future Awareness Changes Everything***

A system that treats each auction independently will always overpay in late sessions (when good vehicles are scarce) and underbid in early sessions (when it doesn't know how good the session will be).

Phase 6 forecasts market conditions to enable **temporal intelligence**.

### ***Dual Forecasting: LSTM + Statistical Fallback***

The system attempts to train a **multivariate LSTM** as market history accumulates:

```python

class MarketLSTM(nn.Module):

def __init__(self, input_size=6, hidden_size=64, num_layers=2, output_size=4):

6 market signals in → 4 forecast values out

self.lstm = nn.LSTM(input_size, hidden_size, num_layers, dropout=0.3)

self.fc = nn.Linear(hidden_size, output_size)

Input features per timestep:

1. Normalized auction price
2. Competition intensity
3. Vehicle quality signal
4. Whether we won

5. Market regime encoding

6. Session progress

Output forecasts:

1. ``future_regime_hot_prob`` — probability the market heats up
2. ``opportunity_density`` — how many good vehicles are likely coming
3. ``price_trend`` — are prices rising or falling?
4. ``volatility`` — how unpredictable is the market?

If PyTorch isn't available or insufficient history exists, the system falls back to **EMA-based statistical forecasting** — always available, less precise.

How Forecasts Affect Decisions

The forecast output feeds into:

- Phase 3's ``AdaptivePolicyEngine`` (when to tighten/loosen margins)
- Phase 4's Opportunity Cost Node (skip now if good things are coming)
- Phase 5's ``OpportunityPreserverAgent`` and ``MarketObserverAgent``
- Phase 7's capital allocation decisions

A high ``future_regime_hot_prob`` means "prices are about to get expensive — preserve capital and pick spots carefully." A high ``opportunity_density`` means "lots of good vehicles are coming — don't blow the bankroll now."

Phase 7 — Meta-Strategic Governance

The Chief Investment Officer

Every professional fund has someone whose job is to look at the entire portfolio and ask: **is this sustainable? Are we taking too much risk? Are we in the right segments?**

Phase 7's ``MetaStrategicGovernor`` does exactly that — for the auction ecosystem.

Capital Allocation by Segment

The governor maintains target capital allocations:

```
```python
capital_allocation = {
 "economy": 20%,
 "mid_range": 45%,
 "luxury": 30%,
 "exotic": 5%,
}
```

These aren't fixed. Every 20 auctions, the governor rebalances based on which segments have been most profitable recently.

### ***Pre-Auction Directives***

Before every auction, the governor issues a directive to the rest of the system:

```
```python
directive = {
    "strategy_mode": "balanced" | "conservative" | "aggressive",
    "allocated_capital": how much of bankroll is available for this segment,
    "max_bid_cap": hard ceiling (80% of segment allocation),
    "emergency_conservation": True if bankroll < 25% of starting,
    "ecosystem_health": 0.0–1.0 composite health score,
}
```
```

The `strategy\_mode` directly feeds into Phase 2's `mode\_scale` parameter, adjusting all bids system-wide.

### ***Strategy Rotation***

The governor continuously evaluates whether the current strategy is working:

- Recent win rate < 30% AND bankroll declining → switch to `conservative`
- Win rate > 60% AND bankroll > 70% of starting → allow `aggressive`
- Everything in between → `balanced`

Strategy changes are logged. The rotation history is maintained for debugging and analysis.

### ***Emergency Conservation Mode***

If the bankroll drops below 25% of starting (\$125,000), the governor triggers emergency conservation:

- All bids are either skipped or capped at minimal amounts
- Only the highest-conviction opportunities (>0.85) are considered
- The goal is survival, not profit

This is the circuit breaker that prevents the agent from going bust.

### ***Ecosystem Health Score***

The governor computes a composite health signal (0.0–1.0) using:

- Recent win rate (weighted 30%)
- Profit rate per auction (weighted 30%)
- Bankroll ratio vs. starting (weighted 25%)
- Market outlook from Phase 6 (weighted 15%)

This score flows into Phase 5's agents as context — agents behave differently when ecosystem health is low.

---

## **The Master Agent — `BiddingAgent`**

The final `BiddingAgent` class is what the auction arena instantiates. It wires all six phases together.

```
`__init__`
```

```
```python
```

```
def __init__(self):
```

Load Phase 1 artifacts

```
model = CatBoostRegressor()
```

```
model.load_model("model_VITIKA.cbm")
```

```
encoders = pickle.load(open("encoders_VITIKA.pkl", "rb"))
```

Initialize all phases

```
self.memory = EpisodicMemory(max_episodes=500) # Phase 3
```

```
self.policy = AdaptivePolicyEngine() # Phase 3
```

```
self.forecast = FutureMarketIntelligence() # Phase 6
```

```
self.reasoning = StrategicReasoningPipeline(...) # Phase 4
```

```
self.ecosystem = MultiAgentEcosystem(...) # Phase 5
```

```
self.governor = MetaStrategicGovernor(500_000) # Phase 7
```

```
self.bidder = EliteBiddingEngine() # Phase 2
```

```
```
```

### ***`analyze\_item(car\_dict)` — Single-Row Preprocessing***

This method applies all Phase 1 preprocessing to a single vehicle dictionary. No `df.mean()`, no `df.groupby()` — every transformation uses only the saved encoder maps.

Key steps:

1. Lowercase all string fields
2. Compute ``car_age``, ``age_bucket``
3. Flag ``is_luxury``, ``is_preferred_color``, ``is_preferred_interior``
4. Apply saved target encoding maps for ``make_score``, ``model_score``, etc.
5. Clip odometer to training bounds
6. Compute ``mileage_per_year``, ``condition_bucket``, composite scores
7. Align to exact ``feature_columns`` order the model expects
8. Return feature dict ready for prediction

### ***`place\_bid(car, current\_highest\_bid, round\_number)` — The Full Decision Loop***

```
```python
```

```
def place_bid(car, current_highest_bid, round_number):
```

1. Get governor directive

2. Predict value (Phase 1)

3. Build AuctionContext

4. Run reasoning pipeline (Phase 4)
5. Run multi-agent deliberation (Phase 5)
6. Compute bid via sigmoid-decay (Phase 2)
7. Apply bankroll guards
8. Return bid amount (0.0 = pass)

...

The key constraint: this method receives one car at a time as a dictionary. It must return a float. No side effects on external state. All intelligence happens inside.

Key Design Constraints

These were non-negotiable requirements from the task spec:

Constraint	Why It Matters
`analyze_item()` processes one row at a time	The agent runs in a live simulation — no batch operations
No `df.mean()` or `df.groupby()` inside inference	Those require the full training dataset
All encoders saved in `__init__`	Preprocessing must be deterministic and instant
Relative paths for model files	Arena environment can't use absolute paths
Bidding is a pure function of `self.predicted_value`, `self.bankroll`, `current_highest_bid`, `round_number`	Deterministic bid sequence requirement

|---|---|

Files Required at Runtime

File	Source	Purpose
`model_VITIKA.cbm`	Phase 1 notebook	CatBoost price prediction model
`encoders_VITIKA.pkl`	Phase 1 notebook	All preprocessing maps and configs

|---|---|---|

Both must be in the same directory as the agent script (relative path loading).

Submission Format

...

YourName_IronLM.zip

■■■ agent_VITIKA.py ← The BiddingAgent class

■■■ model_VITIKA.cbm ← Trained CatBoost model

■■■ encoders_VITIKA.pkl ← Preprocessing artifacts

■■■ analysis_VITIKA.ipynb ← Phase 1 notebook

■■■ report_VITIKA.pdf ← Written report

...

Summary of All Phases

| Phase | What It Does | Key Innovation |

|---|---|---|

| **Phase 2** | Computes actual bid amounts | Sigmoid-decay curves prevent overbidding |

| **Phase 3** | Learns from past auction outcomes | Policy updates from reward signals |

| **Phase 4** | Reasons through the decision | Multi-node pipeline with full audit trail |

| **Phase 5** | 5 agents vote on every decision | RiskAverseAgent has hard veto power |

| **Phase 6** | Forecasts future market conditions | LSTM + EMA fallback for temporal awareness |

| **Phase 7** | Governs the entire ecosystem | Capital allocation + emergency circuit breaker |

Part of the Autonomous Auction Intelligence Ecosystem | Task 1 — Pawn Stars Bidding Challenge

Section 3

■ Adaptive AI Fitness & Nutrition MAS

****A story of how a team of AI agents became your smartest personal trainer****

...

> Imagine having a personal trainer, a dietitian, a sports scientist, a recovery coach, and a safety officer — all working together, *just for you*, every single day. That is exactly what this project builds. Welcome to the ****Adaptive Fitness Multi-Agent System (MAS)****.

■ The Story — What Is This Project?

Meet **Vitika**. She is 27, wants to lose fat, has a shoulder injury, only got 5.5 hours of sleep last night, and feels a fatigue of 7 out of 10. She opens an app and types in her details.

Behind the scenes, ****nine AI agents**** spring to life — each one a specialist. They talk to each other, share findings, argue about safety, and within seconds produce a ***personalized*** workout plan, a ***custom*** nutrition plan, and a ***recovery protocol*** — all adapted specifically to Vitika's state ***today***, not just her general profile.

This is not a static fitness app. This is a **living, breathing, learning system**. Every day Vitika logs in, the system remembers what happened yesterday, learns what works for her, and quietly adjusts. It gets smarter over time.

That is the big idea. Now let's walk through exactly how it is built, phase by phase.

— — —

■■ Project Files

File	What It Does
1. <code>main.c</code>	Contains the <code>main</code> function, which is the entry point of the program. It includes the <code>stdio.h</code> header and calls the <code>add</code> function from <code>add.c</code> .
2. <code>add.c</code>	Contains the <code>add</code> function, which takes two integers as input and returns their sum.
3. <code>add.h</code>	Contains the function prototype for <code>add</code> , which is used by <code>main.c</code> to know the function's signature.

$$| \quad - \quad - \quad | \quad - \quad - \quad |$$

```
| `mas_agent_VITIKA.ipynb` | The entire brain — all 9 agents, the database, the pipeline, the demo  
run |
```

| `flowchart-vitika.ipynb` | Generates beautiful architecture diagrams of the system using Graphviz |

— — —

■■ System Architecture — The Big Picture

Before diving into the code, here is how all the layers connect:

...

USER → Streamlit UI → FastAPI Backend → LangGraph Workflow Engine

↓

[illegible]

■ Profile → Memory → Learning → Retrieval → Workout ■

■ → Nutrition → Recovery → Safety → Orchestrator■

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100 101 102 103 104 105 106 107 108 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233 234 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287 288 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305 306 307 308 309 310 311 312 313 314 315 316 317 318 319 320 321 322 323 324 325 326 327 328 329 330 331 332 333 334 335 336 337 338 339 340 341 342 343 344 345 346 347 348 349 350 351 352 353 354 355 356 357 358 359 360 361 362 363 364 365 366 367 368 369 370 371 372 373 374 375 376 377 378 379 380 381 382 383 384 385 386 387 388 389 390 391 392 393 394 395 396 397 398 399 400 401 402 403 404 405 406 407 408 409 410 411 412 413 414 415 416 417 418 419 420 421 422 423 424 425 426 427 428 429 430 431 432 433 434 435 436 437 438 439 440 441 442 443 444 445 446 447 448 449 450 451 452 453 454 455 456 457 458 459 460 461 462 463 464 465 466 467 468 469 470 471 472 473 474 475 476 477 478 479 480 481 482 483 484 485 486 487 488 489 490 491 492 493 494 495 496 497 498 499 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841 842 843 844 845 846 847 848 849 850 851 852 853 854 855 856 857 858 859 860 861 862 863 864 865 866 867 868 869 870 871 872 873 874 875 876 877 878 879 880 881 882 883 884 885 886 887 888 889 890 891 892 893 894 895 896 897 898 899 900 901 902 903 904 905 906 907 908 909 910 911 912 913 914 915 916 917 918 919 920 921 922 923 924 925 926 927 928 929 930 931 932 933 934 935 936 937 938 939 940 941 942 943 944 945 946 947 948 949 950 951 952 953 954 955 956 957 958 959 960 961 962 963 964 965 966 967 968 969 970 971 972 973 974 975 976 977 978 979 980 981 982 983 984 985 986 987 988 989 990 991 992 993 994 995 996 997 998 999 1000 1001 1002 1003 1004 1005 1006 1007 1008 1009 1010 1011 1012 1013 1014 1015 1016 1017 1018 1019 1020 1021 1022 1023 1024 1025 1026 1027 1028 1029 1030 1031 1032 1033 1034

↓

PERSONALIZED ADAPTIVE PLAN

(Workout + Nutrition + Recovery + Safety Report)

...

Every agent talks to **external APIs** (real exercise data, real food nutrition data, real research) and a **local database** (your history, your memory). Nothing is made up. Everything is personalized.

■ PHASE 0 — Setting Up the World

Cells 1, 2, 3 — "Before the agents can think, they need tools"

Step 1 — Installing the Toolkit (Cell 1)

Think of this as unpacking the toolbox before building a house. The system needs:

- **LangGraph & LangChain** — the orchestration framework that connects all agents
- **Google Gemini AI** — the brain that powers the thinking inside each agent
- **FAISS** — a lightning-fast memory system that finds similar past experiences
- **Sentence Transformers** — converts text into mathematical vectors for smart searching
- **FastAPI & Streamlit** — the web layer so humans can interact with the system

```
```python
```

```
!pip install -q langgraph langchain langchain-google-genai
```

```
!pip install -q faiss-cpu sentence-transformers numpy
```

```
!pip install -q fastapi uvicorn streamlit
```

```
```
```

Step 2 — Importing Everything & Starting the Logger (Cell 2)

Once tools are installed, the system imports them all and boots up a **logger** — a watchful narrator that writes a timestamped diary of everything the system does. Every agent action, every API call, every decision gets logged with a line like:

```
```
```

```
2024-01-15 10:32:01 | INFO | AdaptiveFitnessMAS | [ProfileAgent] Analyzing user profile for demo_user
```

```
```
```

This is invaluable for debugging and understanding what happened.

Step 3 — API Keys & Constants (Cell 3)

Four external services power this system. Each needs an API key:

| Service | What It Does |
|---------|--------------|
|---------|--------------|

| | |
|-----|-----|
| --- | --- |
|-----|-----|

| | |
|---------------------------|---|
| Gemini (Google AI) | The reasoning engine — thinks, plans, writes JSON |
|---------------------------|---|

| | |
|-------------------|--|
| ExerciseDB | A database of 1,300+ real exercises with body-part filtering |
|-------------------|--|

| | |
|----------------------|---|
| USDA FoodData | The US Department of Agriculture's nutrition database — real macro data |
|----------------------|---|

| | |
|---------------|--|
| Tavily | A research search engine — finds relevant fitness studies and injury protocols |
|---------------|--|

Constants like the database path, embedding model name, and LLM model are also defined here. The system uses `gemini-1.5-flash` for speed and efficiency.

> **Fallback Design Philosophy:** If any API key is missing or an API fails, every client has a built-in fallback with hardcoded sensible data. The system never crashes. It degrades gracefully.

■ PHASE 1 — Building the Foundation

Cells 4, 5, 6 — "The skeleton, the database, and the memory"

Step 4 — The Shared State Schema (Cell 4)

This is the **nervous system** of the entire project. Every agent reads from and writes to a single shared `AgentState` dictionary. Think of it as a baton in a relay race — each runner adds their results and passes it forward.

...

AgentState contains:

user_input ← What the user typed

profile ← Structured user profile (age, goal, injuries...)

memory_context ← What we remember from past sessions

learned_preferences ← How the system has adapted to this user

relevant_exercises ← Exercises fetched from ExerciseDB

relevant_meals ← Foods fetched from USDA

workout_plan ← The generated workout

nutrition_plan ← The generated meal plan

recovery_plan ← The recovery protocol

safety_flags ← Any risk warnings

final_plan ← The complete unified output

...

Every agent receives the full state and returns an updated version. No agent talks directly to another — they only communicate through this shared state. Clean, modular, testable.

Step 5 — The Database Layer (Cell 5)

The `DatabaseManager` class manages a **SQLite database** with four tables:

`users` — stores your permanent profile: age, weight, height, goal, injuries, dietary preferences. This is upserted (insert or update) every time you run.

`daily\_logs` — stores your daily check-ins: fatigue level, sleep hours, soreness areas, whether you completed your workout. This is *the key to adaptation* — the system learns from these logs.

`plan\_history` — stores every plan ever generated for you, with a feedback score. The system can look back and see what kinds of plans you responded well to.

`memory\_vectors` — stores the text IDs of memories saved in the FAISS vector index, linking the database to the vector memory.

This database persists between sessions. It is how the system remembers you.

Step 6 — Vector Memory with FAISS (Cell 6)

Here is where things get truly intelligent. The ``VectorMemory`` class uses **FAISS** (Facebook AI Similarity Search) to build a **semantic memory store**.

How does it work?

1. Past plans and logs are converted into **384-dimensional numerical vectors** using a SentenceTransformer model
2. These vectors are stored in a FAISS index
3. When you run a new session, the system searches this index for the **most semantically similar** past experiences
4. It retrieves the top-5 most relevant memories to inject as context

So if Vitika previously had a great week doing moderate-intensity shoulder-friendly workouts, and today her profile is similar, the system will **find that pattern** and lean on it. This is long-term contextual intelligence, not just rule-following.

■ PHASE 2 — Connecting to the Real World

Cells 7, 8 — "The agents can't work in a vacuum"

Step 7 — External API Clients (Cell 7)

Three specialized API clients are built here, each with bulletproof fallback logic:

ExerciseDBClient — Fetches real exercises filtered by body part (chest, back, legs, cardio, shoulders). If the API is unavailable, it falls back to a curated hardcoded library. Every exercise includes the name, equipment needed, target muscle, and body part.

USDAClient — Looks up real macro nutrition data (calories, protein, carbs, fat per 100g) for any food. Falls back to a hardcoded table of common foods like chicken breast, oats, salmon, greek yogurt, tofu, and lentils.

TavilyClient — Searches the web for research relevant to the user's injuries and goals. For example, if someone has a knee injury, it searches for "safe exercise and nutrition for knee injury recovery." Returns up to 3 research snippets to inform the agents.

Step 8 — The LLM Client & Singletons (Cell 8)

The ``llm_call()`` function is the **universal thinking interface** used by every agent. It takes a system prompt (the agent's role and instructions) and a user prompt (the specific data and question), calls Gemini, and returns a text response.

All shared resources — the database, vector memory, and three API clients — are instantiated **once** as singletons. Every agent imports and reuses these same instances. No redundant connections, no memory waste.

■ PHASE 3 — The Nine Agents

Cells 9–17 — "The team assembles"

Each agent is a Python function that receives the full ``AgentState``, does its specialized work, and returns an updated state. Here is the story of what each one does:

■■ ***Agent 1 — Profile Agent (Cell 9)***

****Job:**** Understand who you are.

When Vitika submits her details, this agent is the first to act. It takes the raw unstructured text input and sends it to Gemini with a very specific instruction: *"Extract a structured JSON profile with these exact fields."*

The output is a clean, validated dictionary with:

- Demographics (age, weight, height, gender)
- Goal (fat_loss / muscle_gain / endurance / flexibility / general_fitness)
- Fitness level (beginner / intermediate / advanced)
- Dietary preferences and allergies
- Injuries (critical for safety downstream)
- Today's stats (fatigue level, sleep hours)
- Session preferences (duration, days per week, equipment available)

If parsing fails, it falls back to the existing saved profile from the database. The profile is then upserted (saved or updated) to SQLite.

■ **Agent 2 — Memory Agent (Cell 10)**

****Job:**** Remember the past.

This agent digs into history. It:

1. Fetches the last 14 days of daily logs from SQLite
2. Searches the FAISS vector store for semantically similar past plans
3. Calculates behavioral patterns (average fatigue, sleep trends, workout completion rate)
4. Formats all of this into a ``memory_context`` string and ``historical_patterns`` list

The result is something like: *"Over the last 2 weeks, Vitika averaged fatigue 6.5/10, sleep 5.8h, completed 70% of workouts."* This context flows into every downstream agent.

New memories from past plans are also added to the FAISS index here, so the memory continuously grows.

■ **Agent 3 — Adaptive Learning Agent (Cell 11)**

****Job:**** Decide how hard to push today.

This is the ****intelligence layer**** of the system. It implements a simplified ****contextual bandit algorithm**** to score today's readiness and decide the appropriate training intensity.

The readiness score formula:

```
readiness = (sleep_hours / 9.0) × 0.35
+ (1 - fatigue_level / 10.0) × 0.45
+ completion_rate × 0.20
---
```

Based on the score:

- 0.70+ → ****High intensity****
- 0.50–0.70 → ****Moderate intensity****

- Below 0.50 → **Low intensity / active recovery**

It also applies **injury adaptation rules**. For example, if the user has a shoulder injury, it adds rules like "avoid overhead pressing" and "substitute with horizontal pushing movements." These rules propagate to the Workout Agent.

The output is a `learned_preferences` dictionary: the intensity level, training volume, recovery priority, and specific injury adaptations.

■ **Agent 4 — Semantic Retrieval Agent (Cell 12)**

Job: Fetch real data from the world.

Now that the system knows *who* Vitika is and *how hard* to push her today, it needs actual content to work with.

This agent makes three parallel data calls:

1. **ExerciseDB** — fetches exercises for each relevant muscle group (filtered by goal). Importantly, it *excludes* body parts affected by the user's injuries. If Vitika has a shoulder injury, shoulder exercises are filtered out before the workout agent even sees them.
2. **USDA** — fetches macro nutrition data for goal-appropriate foods. For fat loss it focuses on high-protein foods; for endurance it prioritizes carbohydrate-rich options.
3. **Tavily Research** — if injuries are present, it searches the web for research-backed protocols. This injects real scientific context into the plan generation.

■ **Agent 5 — Workout Agent (Cell 13)**

Job: Build the weekly workout.

Armed with real exercises, the user profile, and the adaptive preferences, this agent calls Gemini with a detailed prompt asking for a structured weekly schedule — each day has a focus, a list of exercises with sets, reps, rest periods, and notes, plus warmup and cooldown routines.

If Gemini's JSON is malformed (which can happen), the system falls back to a safe default bodyweight plan. The plan always comes out.

■ **Agent 6 — Nutrition Agent (Cell 14)**

Job: Calculate what to eat and how much.

This agent does real nutritional science, not guesswork.

Step 1 — Calculate BMR using the Mifflin-St Jeor equation:

...

$$\text{BMR} = 10 \times \text{weight} + 6.25 \times \text{height} - 5 \times \text{age} + (5 \text{ if male, } -161 \text{ if female})$$

...

Step 2 — Calculate TDEE (Total Daily Energy Expenditure) by multiplying BMR by an activity multiplier based on fitness level.

Step 3 — Adjust for goal:

- Fat loss: subtract 400 calories
- Muscle gain: add 300 calories
- Endurance: add 100 calories

****Step 4 — Allocate macros**** using goal-specific ratios (e.g., fat loss = 40% protein, 35% carbs, 25% fat).

****Step 5 — Call Gemini**** to generate a full daily meal plan with Breakfast, Lunch, Snack, and Dinner — each with specific foods, quantities, and per-meal macro totals. Dietary preferences and allergies are respected.

■ **Agent 7 — Recovery Agent (Cell 15)**

****Job:**** Make sure Vitika doesn't overtrain.

This agent takes fatigue, sleep data, training history, and the injury research context and generates a recovery protocol:

- A ****readiness score**** (0–10)
- A ****recovery priority level**** (low / moderate / high / critical)
- ****Sleep recommendations**** with specific tips
- ****Active recovery activities**** (e.g., 20-minute light walk)
- ****Mobility work**** (specific stretches)
- ****Stress management**** techniques (e.g., box breathing)
- A ****next session recommendation**** (proceed / rest / light activity)

If Vitika's fatigue is 7/10 and she slept 5.5 hours, this agent will flag HIGH recovery priority and recommend lighter activity.

■ ■ **Agent 8 — Safety Agent (Cell 16)**

****Job:**** Be the last line of defense.

This is the most critical agent. It audits the entire workout plan against Vitika's injuries using two layers of protection:

****Layer 1 — Rule-based checks:**** A hardcoded risk dictionary maps injuries to dangerous exercises:

- Shoulder injury → flag overhead press, arnold press, upright row, dips
- Knee injury → flag deep squat, jump squat, lunge, leg extension
- Back injury → flag deadlift, good morning, hyperextension, sit-ups
- Wrist injury → flag barbell curls, wrist curls
- Elbow injury → flag skull crushers, tricep dips

****Layer 2 — LLM safety audit:**** The workout is sent to Gemini with the role of a sports medicine safety officer. This catches subtle risks the rules might miss.

Any flagged exercises are surfaced as safety warnings. If there are CRITICAL warnings, `safety\_approved` is set to `False`, which the Final Orchestrator uses to add prominent safety disclaimers and substitutions.

■ **Agent 9 — Final Orchestrator (Cell 17)**

****Job:**** Bring it all together into one coherent plan.

This is the master synthesizer. It takes every single output from every agent and calls Gemini one final time with the complete picture — asking it to synthesize everything into ONE unified adaptive plan, explain WHY today's plan differs from a standard plan, and include safety warnings and substitutions if needed.

The final output contains:

- A human-readable `user\_summary` and `today\_recommendation`
- An `adaptation\_rationale` explaining the personalization decisions
- The complete `workout`, `nutrition`, and `recovery` plans
- All `safety\_flags` with substitution suggestions
- `progress\_metrics` to track over time

This plan is saved to the SQLite database for future memory.

■■ PHASE 4 — Wiring It All Together

Cells 18, 19 — "The pipeline becomes one"

Step 18 — Building the LangGraph Workflow (Cell 18)

LangGraph is the **conductor** that orchestrates all nine agents. The `build_graph()` function registers all nine agents as nodes, adds directed edges connecting them in sequence, and compiles the graph into an executable workflow.

The flow is strictly sequential:

...

```
profile_agent → memory_agent → adaptive_learning_agent → semantic_retrieval_agent
→ workout_agent → nutrition_agent → recovery_agent → safety_agent
→ final_orchestrator → END
```

...

This is the relay race in action. Each agent hands the baton to the next, enriching the shared state along the way.

Step 19 — The Pipeline Runner & Daily Logger (Cell 19)

Two utility functions power the system's public interface:

`run_pipeline(user_id, user_input)` — The main entry point. Takes a user ID and their input dictionary, builds the graph, creates the initial empty state, invokes the full pipeline, and returns the final adaptive plan.

`log_user_day(user_id, fatigue, sleep_hours, ...)` — Lets users log their daily state. This feeds the Adaptive Learning Agent. The more consistently a user logs, the smarter the system becomes about their patterns.

■ PHASE 5 — Running the Demo

Cells 20, 21 — "See it in action"

Step 20 — The Demo Run (Cell 20)

This cell brings Vitika to life as a sample user. It:

1. Defines her full profile (27F, 68kg, fat loss goal, shoulder injury, vegetarian, fatigue 7/10, sleep 5.5h)
2. Logs 3 days of historical daily data to seed the memory
3. Runs the full 9-agent pipeline
4. Prints a summary of the adaptive plan to the console
5. Saves the complete plan to `demo\_adaptive\_plan.json`

Sample output:

...

ADAPTIVE PLAN GENERATED

Summary: Personalized plan for Priya Sharma — fat loss focus

Today's Rec: Low-Moderate intensity (high fatigue detected)

Key Adaptations: ['No shoulder overhead movements', 'Fatigue-adjusted volume']

Safety Flags: 2 flag(s)

...

Step 21 — (Optional) Streamlit UI Launch (Cell 21)

For those who want a visual interface, this cell starts a Streamlit web server and creates a public ngrok URL to access the UI from anywhere — giving the system a real front-end with a profile form, plan display, chat interface, and progress tracker.

■ PHASE 6 — The Architecture Diagrams

*`flowchart-vitika.ipynb` — "Visualizing the machine"

This separate notebook generates two professional architecture diagrams using **Graphviz**.

Diagram 1 — Full System Architecture

A top-down view showing all six layers — User → Frontend → Backend → Orchestration → 9 Agents → Data APIs → Output — color-coded by role.

Diagram 2 — LangGraph Agent Execution Flow

A detailed flowchart showing exactly what each agent receives as input, what it processes, and what it outputs — with arrows labeled with the state variables being passed. Includes the **memory feedback loop** (a dashed red arrow from the Final Orchestrator back to the Memory Agent) showing how the system learns from each completed session.

To generate both diagrams:

```
```bash
```

```
pip install graphviz
```

```
apt-get install graphviz # Linux
```

```
brew install graphviz # Mac
```

```
jupyter nbconvert --to notebook --execute flowchart-vitika.ipynb
```

```
```
```

Outputs: `adaptive\_architecture.png` and `langgraph\_flow.png`

■ How to Run This Project

Prerequisites

```
```bash
pip install langgraph langchain langchain-google-genai
pip install google-generativeai requests python-dotenv
pip install faiss-cpu sentence-transformers numpy
pip install fastapi uvicorn streamlit
```
```

API Keys Needed

Get free keys from:

- Gemini: <https://aistudio.google.com/app/apikey>
- ExerciseDB: https://rapidapi.com/justin-WFnsXH_t6/api/exercisedb
- USDA: <https://fdc.nal.usda.gov/api-guide.html>
- Tavily: <https://tavily.com>

Paste them in Cell 3 of `mas\_agent\_VITIKA.ipynb`.

Run the Demo

Open `mas\_agent\_VITIKA.ipynb` in Jupyter and run all cells top to bottom. The demo in Cell 20 runs automatically. You will see the full pipeline execute and a complete adaptive plan saved to `demo\_adaptive\_plan.json`.

■ Tech Stack Summary

| Component | Technology |

|---|---|

| Agent Orchestration | LangGraph |

| LLM / AI Reasoning | Google Gemini 1.5 Flash |

| Embeddings / Semantic Memory | Sentence Transformers (all-MiniLM-L6-v2) |

| Vector Search | FAISS |

| Persistent Storage | SQLite |

| Exercise Data | ExerciseDB API |

| Nutrition Data | USDA FoodData API |

| Research Intelligence | Tavily Search API |

| Frontend | Streamlit |

| Backend API | FastAPI |

| Architecture Diagrams | Graphviz |

■ Key Design Principles

****Adaptation over prescription**** — the system doesn't give the same plan twice. Every session is recalculated based on how you actually felt yesterday, not how you felt on Day 1.

****Safety first**** — two layers of safety checks (rule-based + AI) ensure that injury-sensitive users never receive dangerous exercise recommendations.

****Graceful degradation**** — every external API call has a fallback. The system always produces a complete plan, even if every API is down.

****Memory as intelligence**** — FAISS vector memory means the system genuinely gets smarter the longer you use it. It is not just storing data; it is finding meaningful patterns.

****Modular agents**** — each agent is a self-contained function. You can swap out the workout agent, change the nutrition logic, or add a new agent without touching the others.

Built with LangGraph, Gemini, FAISS, and a deep belief that fitness AI should adapt to you — not the other way around.